

פרק 2: בדיקות לאורך מחזור הפיתוח – מודלים, רמות, סוגים ו-Agile

פרק A – מודלי פיתוח

• **Sequential development** (פיתוח סדרתי): מודל קלאסי כמו ה-Waterfall, שבו השלבים (דרישות → עיצוב → מימוש → בדיקות) נעשים אחד אחרי השני. בבדיקות סדרתי, מרבית הבדיקות – ובמיוחד בדיקות מערכת וקבלה – מתבצעות רק בסוף. רמת K2: זויות מבחן: איך המבנה הסדרתי משפיע על מתי מבצעים בדיקות (למשל מרוכזות בסוף). טעות שכיחה: להניח שבשל הדרגתי אין כלל בדיקות עקב (לדוגמה "בדקנו ביחידה אז אין צורך בבדיקת מערכת").

• **Iterative/Incremental development** (פיתוח איטרטיבי/אינקרמנטלי): בגישה זו התוכנה מתפתחת במחזורים חוזרים (sprints/איטרציות); בכל מחזור מוסיפים מודולים או תכונות (increment) ומשלבים אותן במערכת. בדיקות מבוצעות בכל איטרציה על החלקים החדשים, וניתן לתכנן בדיקות באופן מחזורי [L198-L207+45]. רמת K2: זויות מבחן: השפעת הפיתוח המחזורי על רצף הבדיקות (בדיקות מוקדמות וגלובליות כלפי משתמש). טעות שכיחה: לחשוב שבגישה איטרטיבית אפשר לדלג על דוקומנטציה של בדיקות כי "יודעים הכול".

• **V-Model** (מודל V): הרחבה של מודל הסדרה המעמיקה את תיאום הבדיקות עם הפיתוח. בצד השמאלי (ה-V) של המודל: פעילות פיתוח (דרישות, עיצוב, קוד) ובצד ימין מתבצעות הבדיקות התואמות: בדיקות יחידה ברמת הקוד, בדיקות אינטגרציה, בדיקת מערכת ו-קבלה. כלומר, לכל שלב פיתוח מוקצה שלב בדיקה מקביל באותו העומק [L74-L81+49] [L49+L47-L54]. רמת K2: זויות מבחן: הקשר בין פעילויות בדיקה לפיתוח ויתרונות (למשל שמירת עקיבות ובדיקות מוקדמות). טעות שכיחה: להניח שבדיקת קבלה שייכת רק לסוף המודל ולהתעלם מההיקשרות לתכנון מערכתי מוקדם.

• **Agile** (אג'יל): גישה גמישה ואינקרמנטלית שמקדמת צוותים חוצי תפקידים עם משוב תדיר מהלקוח. הפיתוח באג'יל מתבצע באיטרציות קצרות (Sprint), ובכל איטרציה מקבל המוצר תכונות חדשות ונבדקות שוב הפעולות הקודמות. ערכי ההנחה כוללים שיתוף פעולה מתמשך עם הלקוח, היכולת להגיב לשינויים ותיעדוף עמידה בדרישות המשתמש. רמת K2: זויות מבחן: איך מהירות השינויים והגמישות באג'יל מחייבות ביצוע בדיקות מוקדם (פיתוח מונחה בדיקה) ובשיתוף מלא של הבודק. טעות שכיחה: להניח ש-Agile פירושו שאין תכנון או מסמכים כלל לבדיקות ("אונבקום" – work backward).

• **בדיקות ב-DevOps** (שילוב בדיקות בפיתוח ותפעול): DevOps מאחד בין צוות הפיתוח וצוות התפעול (כולל QA) כדי לאפשר אינטגרציה רציפה ופיתוח מתמשך (CI/CD). לכן, בבדיקה כחלק מ-DevOps מריצים בדיקות אוטומטיות (unit, אינטגרציה, מערכת) בכל קומיט קוד [L264-L272+46]. רמת K2: זויות מבחן: תיאום בין בדיקות ואוטומציה לפעילויות הפצה רציפה. טעות שכיחה: לראות ב-DevOps רק אוטומציית תשתית ולהמעיט בחשיבות הבדיקות בסביבה משלבת (לדוגמה "אוטומציה תופסת את כל האגים").

• **Shift-left testing** (בדיקות מוקדמות): מושג של הזזת הבדיקות לשלב מוקדם יותר במחזור הפיתוח. כלומר, לא מחכים לשלב מערכת/קבלה אלא מבצעים בדיקות כבר ברמות התחונות (יחידה/אינטגרציה) [L277+46]. היתרון: איתור פגמים מוקדם יותר חוסך עלויות תיקון מאחר שתיקון באגים בקטע מוקדם במחזור זול יותר ובעל סיכוי נמוך להשלכות שליליות מאוחרות [L100-L109+42]. רמת K2: זויות מבחן: עלויות השהיית בדיקות והיתרונות שבגילוי מוקדם [L100-L109+42]. טעות שכיחה: לחשוב ש-shift-left מוריד את כמות הבדיקות הכלליות (הוא מזיז אותן קדימה אך לא מבטל בדיקות מערכת/קבלה).

• **Retrospective** (רטרופקטיבה): פגישה מבנית בסוף כל איטרציה או שלב בפרויקט Agile, שבה הצוות (מפתחים ובודקים יחד) דן במה הצליח ומה דורש שיפור בתהליך העבודה [L288-L297+46]. רמת K2: K. זווית מבחן: בדיקת חשיבות ושימוש ברטרופקטיבה לאיתור תקלות בתהליך ולא רק במוצר. טעות שכיחה: לקיים רטרופקטיבה אבל להתמקד רק ברשימת באגים, במקום בשאלות תהליך ("מה עבד טוב? מה נשאר? מה לשפר?") [L288-L297+46].

פרק B – רמות בדיקה

• **בדיקת יחידה (Component/Unit Testing)**: בדיקה של היחידות הקטנות ביותר (מודול, פונקציה, מחלקה) על ידי המפתח או בודק יחידות. אובייקט הבדיקה הוא קוד בודד; מבוצעת תוך גישה לקוד (white-box) בדרך כלל. תופסת באגים לוגיים, שגיאות בתחביר והנחות שגויות על משתנים מקומיים – דברים שלא יתפסו בבדיקות אחרות ברמות גבוהות יותר. רמת K2: K. זווית מבחן: אילו באגים או קוד רלוונטיים יתגלו כאן אבל לא בבדיקות אינטגרציה (למשל Loop שקופץ?). טעות שכיחה: להניח שבדיקת יחידה נכונה פותרת את כל הבאגים ההגיוניים, ולדלג על בדיקות אינטגרציה/מערכת.

• **בדיקת אינטגרציה (Component Integration Testing)**: בדיקה של ממשקים ואינטראקציות בין מודולים/קומפוננטות שנבדקו בנפרד. מבוצעת לרוב בידי בודקי אינטגרציה (שעשויים להיות גם המפתחים) ושואפת לגלות שגיאות בתקשורת בין חלקים (למשל העברת נתונים, קריאות API). אובייקט הבדיקה הוא קבוצות של מודולים שייכות. תופסת באגים שלא נראו בבדיקת יחידה (למשל חוסר התאמת פורמט או שינוי שם שדה). רמת K2: K. זווית מבחן: אבחנה בין בדיקות אינטגרציה מרכיבית לבין מערכתית [L39-L43+56]. טעות שכיחה: לבצע בדיקות יחידה מבודקות ולהניח שהמעבר לאינטגרציה בטוח, או להפך – לבדוק את האינטגרציה במקום יחידות בגלל בלבול במונחים.

• **בדיקת מערכת (System Testing)**: בדיקה של כל המערכת המשולבת כולה, בדרך כלל ללא התערבות בקוד (black-box). מבוצעת על ידי צוות בדיקות עצמאי, על סמך דרישות המערכת. אובייקט הבדיקה הוא המערכת בשימוש (כולל ממשק משתמש ונתונים). תופסת באגים ברמת פונקציונליות כוללת, ביצועים, אמינות, אינטגרציה פנימית והיעדר תכונות לפי הדרישות. רמת K2: K. זווית מבחן: אילו בעיות מערכת (כגון בביצועים או בעיות end-to-end) ניתנות לזיהוי כאן ולא בבדיקות אינטגרציה. טעות שכיחה: להסתפק בבדיקות יחידה ואינטגרציה ולהניח ש"אין צורך בבדיקת מערכת נפרדת" [L386-L392+13].

• **בדיקת אינטגרציה מערכתית (System Integration Testing)**: בדיקה של אינטגרציית המערכת עם מערכות חיצוניות או רכיבים חיצוניים (למשל מסדי נתונים, שירותי API חיצוניים, רכיבי חומרה). דומה לבדיקת אינטגרציה, אבל על גבי המערכת המלאה ורחבה. מבוצעת על ידי בודקי מערכת או DevOps לעיתים. אובייקט הבדיקה הוא מערכת שלמה + מרכיבים חיצוניים. תופסת בעיות תקשורת וקונפיגורציה בין המערכת לסביבות אחרות שאינן נחשפות בבדיקת מערכת מבודדת.

• **בדיקת קבלה (Acceptance Testing)**: הבדיקה ברמה העסקית, שמטרתה לוודא שהמוצר עונה לצרכי הלקוח. מבוצעת בסוף בידי לקוח או נציגו – לא על ידי צוות הפיתוח – בסביבה המדמה את השימוש האמיתי. אובייקט הבדיקה הוא המערכת המלאה תוך ראייה עסקית/רגולטורית. תופסת פערים בהבנת דרישות, נושאי חוויית משתמש, תאימות רגולטורית ועוד. רמת K2: K. זווית מבחן: הבדל בין בדיקות קבלה (מניעת הפרעות בקול הקצה) ובין בדיקות מערכת (ע"י QA) [L386-L392+13] [L1400-L1404+64]. טעות שכיחה: לראות ב-UAT רק "בדיקת מערכת נוספת", או לחסוך אותה בטענה ש"המוצר כבר נבדק".

שיוך פגם לרמה: אם פגם מתגלה לראשונה ברמה נמוכה יותר (למשל באינטגרציה של יחידות) – הוא מייחס לבדיקת רמת האינטגרציה (ולא לרמה גבוהה יותר שבה הפגם רק מתגלה שנית).

פרק C – בדיקות קבלה

• **UAT – User Acceptance Testing** (בדיקת קבלת משתמש): בדיקה על ידי משתמש קצה או נציגי הלקוח בסביבה חיצונית/אמיתית, כדי לוודא שהמערכת עונה על צרכי הלקוח והדרישות העסקיות [L1400-L1404+64].

【L386-L392†13】. נערכת בסוף הפרויקט, לפני השקה. הקשר מבחן: בדיקת המעבר לתנאי חיים אמיתיים, ואימות שימושיות ועמידה בדרישות המשתמש. טעות נפוצה: לראות בה העתק פשוט של "בדיקת מערכת" (שעתור פיצ'רים, לוותר על אמיתיות סביבתית), או להניח ש"כולנו משתמשים בהם כך וכך".

• **בדיקת קבלה תפעולית (Operational Acceptance Testing, OAT):** בדיקה המתמקדת בהיבטים התפעוליים של המערכת – אמינות, ביצועים בסביבה קריטית, אינטגרציה עם מערכות קיימות ותהליכי עבודה. למשל: אם למערכת יש תזמון משימות, בנקודת זמן הנחוצה בסביבת PROD. תפקיד: לוודא שהמערכת עומדת במטרות העסקיות, בתרחישים תפעוליים, ובדרישות השרשרת הלוגיסטית 【L440-L447†13】. קבוצה זו של בדיקות בודקת שהמערכת מוכנה לתפעול יומיומי ולניהול סיכונים.

• **בדיקת קבלה חוזית/רגולטורית (Contractual/Regulatory Acceptance):** בדיקות שממוקדות בהבטחת עמידה בהסכמים חיצוניים או בחוקים ותקנים מסויימים. בבדיקת חוזה, מקפידים שהמוצר מקיים באופן מדוייק את ההתחייבויות בספר החוזה ובמסמכי הפרויקט 【L448-L453†13】. בבדיקת רגולציה, בודקים שהמוצר עומד בכל דרישות החוק והתקינה הרלוונטיות לתחום (למשל תקנות פרטיות, נגישות, אבטחה) 【L456-L462†13】. חזוית מבחן: בדיקות אלה שונות מ-UAT רגיל כי הן לפי מסמך מחייב או דרישות חיצוניות, ולא דווקא לפי חוות דעת המשתמש.

• **בדיקות אלפא (Alpha Testing):** שלב ראשוני של בדיקות קבלת משתמש פנימיות (לרוב בידי צוות ה-QA או מפתחים). מבוצעות טרם שחרור לבטא, במצב קרוב לאמיתי אך עדיין בפנים החברה 【L183-L191†18】. אובייקט הבדיקה: גרסת המוצר במצב מאוד קרוב לייצור. תופס: באגים חמורים ביציבות ובפונקציונליות הראשונית שהחברה רוצה לתקן לפני המתן למשתמשי בטא 【L183-L191†18】. חזוית מבחן: בדיקה מוקדמת של חוויית משתמש בסיסית ועצירת באגים קריטיים; בדיקות אלפא כוללות לעתים שני שלבים (חשיפה ראשונית וחזרה) 【L183†18-L191】. טעות נפוצה: לדלג על אלפא בטענה ש"נראה לנו מוגן" או לבלבל בין אלפא לבדיקת יחידה/אינטגרציה (הן נבדקות רק על ידי ה-QA, אך אלפא היא כבר גרסה מעורבת).

• **בדיקות בטא (Beta Testing):** בדיקה חיצונית על ידי קבוצה מוגבלת של משתמשי קצה אמיתיים (מחוץ לארגון), בסביבה אמיתית 【L239-L247†18】. משתמשי הבטא מספקים משוב על יציבות, שימושיות וחווית משתמש שלא תמיד נראים על ידי הבודקים הפנימיים. זוהי הדרך "האמצעית" לפני השקה רשמית. המטרה: לאתר בעיות שהמפתחים והבודקים הפנימיים לא זיהו ולמדוד קבלת המוצר על-ידי קהל יעד 【L239-L247†18】. חזוית מבחן: בדיקת שימוש "חיצוני" ומצבי קצה (edge cases) בסביבת אמת. טעות נפוצה: לחשוב ש-בטא "לא באמת משנה" כי זו רק משוב (בפועל באגים מהותיים שעלו ב-בטא מחייבים תיקון), או להניח ש"הרשיון נגמר", כלומר שהפידבק הוא פחות חשוב.

פרק D – סוגי בדיקות

• **Functional Testing** (בדיקה פונקציונלית): בדיקות שמוודאות שכל תהליכי הפונקציונליות של המערכת עובדים כנדרש לפי הדרישות. בודקים תרחישים עסקיים ותפקוד מערכתי מול דרישות נמסרות (בין אם בבדיקות ידניות או אוטומטיות) 【L246-L254†54】. אובייקט: הכללי של הפונקציונליות. חזוית מבחן: בדיקת הפעולה כנגד התרחיש המוגדר; במבחן שואלת אם פונקציה X אכן מבצעת Y כמתואר בדרישות. טעות נפוצה: לערב תכונות ביצועים בבדיקות פונקציונליות במקום הבחנה בינה לבין non-functional, או להפך – להתעלם מהיבטים פונקציונליים ולבדוק רק "בולי קצה".

• **Non-Functional Testing** (בדיקה לא-פונקציונלית): בדיקות של מאפייני איכות המערכת – ביצועים, אמינות, אבטחה, נגישות, תחזוקתיות וכו'. לדוגמה: בדיקת עומסים, בדיקת פרופיל זיכרון, בדיקת שמישות (usability) או בדיקת עמידה בתקני אבטחה. מודל ISO 25010 מגדיר קבוצת מאפיינים כוללת: פונקציונליות, ביצועים, אבטחה, תאימות, שימושיות, אמינות, תחזוקה, ניידות וכו' 【L27-L30†67】. רמת K2: חזוית מבחן: המשמעות היא כיצד בודקים תכונות כגון זמן תגובה או קיבולת – תרחישי עומס, עמידות לכשל וכד'. טעות נפוצה: לשלב מטלות non-functional בחשיבה על functional (לדוגמה להשתמש באותם בדיקות פונקציונליות למדוד ביצועים), או לפספס בדיקה לא-פונקציונלית לגמרי.

• **Change-related Testing** (בדיקות הקשורות לשינויים): בדיקות שמבוצעות לאחר שינויים במערכת. הסוגים העיקריים הם *retesting* (בדיקת אישור תיקון, Confirmation) – שבו בודקים שהבאג המדווח תוקן כפי שציפנו [L26-L32+72] – ו-*regression testing* (בדיקות רגרסיה) – שבו מריצים שוב סט בדיקות נבחר לאחר שינוי כדי לוודא שהשינויים לא שיבשו פונקציונליות קיימת [L14-L20+75]. רמת K2: K. זויות מבחן: הבחנה בין "בדיקת תיקון ספציפית" ל"בדיקות כלליות חוזרות". טעות נפוצה: להתבלבל בין השניים – למשל לכנות בטעות רגרסיה כקונפירמציה, או לוותר על רגרסיה בטענה ש"זה כבר נבדק" כשהאטרציה שינתה הרבה.

• **Black-box Testing** (בדיקת קופסה שחורה): בדיקה ללא ידע בקוד הפנימי של התוכנה. הבודק/ת מתמקד/ת בקליטים ופקודות ובבודק שהתשומות מובילות לתוצאות הנכונות לפי המפרט [L87-L91+69]. מתבצעת בדרך כלל בשלב בדיקת מערכת או קבלה, ומבוצעת על ידי בודקים. זויות מבחן: התמקדות בהתנהגות החיצונית של המערכת ללא הסתכלות פנימית [L87-L91+69]. טעות נפוצה: להניח שידועים "איך הקוד אמור לעבוד" ולבצע בדיקות שדורשות גישה לקוד, במקום מבחן קופסה שחורה.

• **White-box Testing** (בדיקת קופסה לבנה): בדיקה עם גישה למבנה הפנימי של התוכנה. הבודק/ת משתמש/ת בידע בקוד, בזרימת השליטה ובמבני המידע כדי לבחור תרחישי בדיקה [L101-L106+69]. מתבצעת לרוב על ידי המפתח או בודקי יחידה. זויות מבחן: כיסוי קוד (למשל בדיקת מסלולים, סניפים) ולא רק פונקציות חיצוניות [L101-L106+69]. טעות נפוצה: לחשוב שמבחן לבן "כולל" אוטומטית את כל הפונקציות החיצוניות או להיפך – לנסות לבדוק פונקציונליות גדולות בלי לפתוח את הארכיטקטורה.

• **בדיקת אישור (Confirmation Testing)**: בדיקה (המכונה גם *re-testing*) שבודקת שהבאג שתוקן באמת נפתר [L26-L32+72]. הבדיקות מבוצעות על אותן שלבים ונתונים שבהם באג היה קיים בעת הגילוי המקורי [L26+72]. רמת K2: K. זויות מבחן: פעולה מצומצמת מאוד – הרצת המבחנים הספציפיים לתיקון הדיווח. טעות נפוצה: לא לרוץ על אותו סצנריו באופן מדויק, או לחשוב שבדיקת אישור תסגור את הבאג סופית מבלי לרוץ שוב על כלל המערכת.

• **Regression Testing** (בדיקות רגרסיה): בדיקות שנערכות לאחר שינויים (כגון תיקוני באגים או הוספת פיצ'רים) על מנת לוודא שהתכונות הקיימות עדיין פועלות כמצופה [L14-L20+75]. בבדיקת רגרסיה מריצים מחדש קבוצה נרחבת של מקרי בדיקה (שלרוב נבחרו מראש ב-suit) כדי לגלות בעיות בלתי מכוונות שנגרמו מהשינויים החדשים. רמת K2: K. זויות מבחן: היקף רחב – לא רק לחזור על הבאג שהתקן, אלא על שטחים מערכתיים שעשויים להיפגע (מה שמבדיל אותה מאישור התיקון המצומצם [L14-L20+75]). טעות נפוצה: לוותר על רגרסיה מחשש בזבוז זמן ("אמרתי שעבד, הכל נקי!") או להסתכל רק על הקוד ששונה במקום על המערכת כולה.

פרק E – Agile לבודקת

• **User Story** (סיפור משתמש): תיאור תמציתי של דרישה או תכונה, מנקודת מבט המשתמש, בשפה טבעית. למשל: "כ- <תפקיד> ארצה <יכולת> כדי <תועלת>". מטרת הסיפור היא לתקשר לצוות מה הפונקציונליות והצורך העסקי באופן בהיר וממוקד [L138-L142+95]. רמת K1: K. זויות מבחן: האם הסיפור כולל מרכיבים קריטיים (תפקיד משתמש, יכולת, מטרה) ומגדיר תרחיש ברור לבדיקות. טעות נפוצה: להשאיר סיפור עמום או ארוך מדי (מעבר למספר שורות), או להגזים בפרטים טכניים במקום לשמור על תיאור משתמש.

• **Acceptance Criteria** (קריטריוני קבלה): תנאים ברורים ומדידים שהסיפור צריך לעמוד בהם כדי להיחשב "בוצע". כל קריטריון מגדיר פירוט של תוצאה רצויה או התנהגות צפויה. נהוג לבנותם כך שיהיו **testable** – למשל בפורמט Given-When-Then (נתון-כאשר-אז), שמתאר תרחישים קונקרטיים של בדיקות. רמת K2: K. זויות מבחן: כתיבת ארכיטקטורת בדיקה מגלילה מהקריטריונים; למשל לוודא שכל השלבים, ערכי גבול ותרחישי קצה מתוארים ב-AC. טעות נפוצה: ניסוח AC בלתי מדויק או עמום ("המערכת תפעל היטב" – לא קביל), או לכלול יישות אחת בלבד בתנאי קבלה שאינה בתנאי Given-When-Then.

• **Definition of Ready/Done** (הגדרת מוכן/הגדרת בוצע): מסמכים מוסכמים בקרב הצוות לגבי קריטריונים שמבטיחים שהמשימה (סיפור משתמש) מוכנה לכתיבה ושהיא אכן הושלמה. *הגדרת מוכן* (DoR): רשימת תנאים שמבטיחים שהסיפור ברור, מוסבר, עם נהלים וכלים נדרשים וניתן לבצע אותו [L1471-L1475+103]. *הגדרת*

בוצע (DoD): רשימת קריטריונים לכל מוצר משנה שהצוות דורש כדי לסמן ש"המוצר מסיים" [L1471+80].
רמת K2: K. זויות מבחן: ודאי שכל הסיפור עומד בקריטריונים הללו לפני שבדקים בו; למשל, האם יש לכל תו ב-DoD מבחנים יחידה ובדיקות אינטגרציה? טעות נפוצה: לא להגדיר DoR/DoD רשמית וליישם אינטואיטיבית בלבד, מה שמוביל למשימות פתוחות ולבאגים "שמחליקים תחת השטיח".

• **ATDD (Acceptance Test-Driven Development)**: גישת פיתוח שבה צוות (לקוח, מפתח, בודק) משתף פעולה בכתיבת **מבחני קבלה** עוד לפני כתיבת הקוד [L383-L390+77]. המבחנים משקפים את עיקר הדרישות מנקודת מבט המשתמש ומוכרים גם כהדיון "שלושת החברים" – הלקוח, המפתח והבודק – שבמהלכו מנסחים את המקרים הרצויים [L383-L390+77] [L395-L391+77]. רמת K2: K. זויות מבחן: אופן הפעלת GWT כמבנה ל-acceptance criteria, ואופן מימוש התיאום בין צוותים (ATDD/BDD). טעות נפוצה: להתייחס ל-ATDD כביטוי טכנולוגי בלבד (למשל כלי בדיקה) ולא להבנה לשיחה על הדרישות המשותפת [L383-L390+77].

• **Whole-team approach** (גישה קבוצתית כוללת): עקרון ב-Agile שבו כל חברי הצוות – מפיתוח, בדיקות, ניהול מוצר ואחרים – אחראים יחדיו על איכות המוצר. הבודק אינו מרוחק אלא עובד ממוקדם עם שאר הקבוצה בכל שלב [L83-L92+38]. רמת K2: K. זויות מבחן: הבדלות בין הפרדה מסורתית של QA ואג'יל של "צוות שלם", ושאלה מי מכתוב סטס פלן ומתי (למשל התרחשות בדיקות בפרויקט משותף [L83-L92+38] [L104-L107-38]). טעות נפוצה: לחשוב שהבודק עושה הכל לבד אחרי הפיתוח ("בדיקה כמקצוע מופרד"); למעשה הבודק משתתף ב-planning ויודע את כל ההקשר.

• **תפקיד הבודקת באיטרציה**: בבדיקת Agile הבודקת שותפה בכל שלבי הסיפור מוחות על הסיפורים: בכתיבת ה-User Stories והקריטריונים, בהערכת מאמץ (estimating), ואף בפיתוח (לדוגמה הבדיקה מוקדמת או אוטומציה בסיום ה-iteration). היא מבצעת בדיקות exploratory או אוטומטיות יחד עם המפתחים, ומוודאת שהמוצר הסופי עונה על צורכי הלקוח [L83-L92+38] [L104-L107-38]. רמת K2: K. זויות מבחן: אילו פעולות בודקת מבצעת בכל סבב, לעומת צוות מסורתי (למשל: האם בודקת תהיה במחזור תכנון, האם מריצה ביצועים?). טעות נפוצה: לחשוב שבתור "בודקת Agile" העבודה היא רק לרוץ לבדיקות ידניות בסוף האיטרציה, במקום להיכלל בתכנון באוטומציה מתחילתו.

פרק CI/CD – F לבודקת ידנית

• **צנרת CI/CD**: מערכת אוטומטית המאגדת Build ו-Release: בכל שינוי קוד (commit) מצטרף ה-build, ריצה של בדיקות אוטומטיות (unit, אינטגרציה) ולבסוף פריסה לסביבת בדיקה או פריסה אוטומטית לסביבת ייצור (Continuous Deployment). מבחינת בודק ידני, ה-CI/CD משמש לבצע בדיקות regression אוטומטיות תדירות [L264-L272+46]. רמת K2: K. זויות מבחן: שאלות על איך מבינים תוצאות ריצה בצנרת (האם מדובר בפגם במוצר, בבדיקות עצמן או בסביבה). טעות נפוצה: להתייחס לתוצאה כ"שאלה לא קשורה" ולדלג על חקר הגורם (למעשה כשיש CI כושל צריך לברור אם זה BUG אמיתי במוצר, SCRIPT שגוי או תצורת סביבה בעייתית).

• **טיראז' של ריצת CI כושלת**: כאשר הצנרת מדווחת כישלון, יש לבצע ניתוח: אם הבדיקות הכתובות נכשלות – האם זה פגם בתוכנה (שצץ בשינוי האחרון), פגם בבדיקת הביצוע (לדוגמה מחריגות בלתי צפויות בקוד הסקריפט), או תקלה בסביבה? לדוגמה, שגיאה באובייקטי UI עשויה להיגרם משינוי ב-HTML (פגם מוצר) או עקב שינוי בתצורת החומרה (תקלה בסביבה). רמת K3: K. זויות מבחן: מיון הטעויות לקטגוריות (BUG vs Test vs Env) ויישום החלטה נכונה. טעות נפוצה: לקרוא לכל כישלון "באג במוצר" מיידית בלי לבדוק (או ההפך – להתעלם מכשל CI כאילו אינו רלוונטי).

טעויות נפוצות (שגיאות שטח)

• **בלבול בין Regression ל-Confirmation**: נוטים לחשוב ששני המושגים זהים ("גם ככה בודקים חוזרים"), אך confirmation נועד לבדוק בדיוק את התיקון, בעוד regression בודק שלא שברנו דבר.
• **"בדקנו ביחידה אז אין צורך במערכת"**: הנחת יסוד מוטעית שרק משום שבאג לא נמצא ב-unit יש צורך או לא בבדיקת מערכת.

- **UAT = System Testing**: לראות ב-UAT רק כבדיקה נוספת במקום חשיבות ממשמש סופי (או להפך – להתעקש שהלקוח ידבר בכללית, ושכל דרישותיו ידועות מראש).
- **בלבול בין Functional/Non-Functional**: למשל לחשוב שבדיקת ביצועים היא סוג של בדיקה פונקציונלית רגילה, או שהבטחה לנגישות איננה בדיקה.
- **בדיקת קבלה חוזית כמו כל בדיקה עסקית רגילה**: לטעות בתנאי נסיגה (בדיקת חוזה היא לפי חוזה ממשי, לא רק קבלה של הלקוח).
- **לדגבל על הגדרת ייעד / לראש**: לדוגמה, סמן את כל הדרישות ב-DoD במקום לוודא שהקוד באמת עבר כיסוי מתאים.
- **מעבר לציפיות מבודק Agile**: להמתין לסוף הספרינט לבדוק במקום לפעול כל הזמן עם המפתחים.
- **התעלמות מסביבת המערכת**: בשלב אינטגרציה או acceptance להניח שהכל בסדר רק כי בסביבת dev רץ בלי שגיאות.
- **“בדיקות exploratory זה לא נחוץ”**: לפעמים ב-Agile שוכחים לערוך בדיקה חווייתית רחבה (exploratory) בין לבין.
- **הערכה מוטעית של RISK**: לדוגמה, לדלג על בדיקות רגרסיה על חשבון תכונות חדשות מפני שהאנשים “בטוחים שעבד”.

מבני תרגיל מקובלים

- **שיוך פגם לרמת בדיקה**: מביאים תיאור פגם (למשל ירידת ביצועים בעת העמסה, או שגיאת NullPointerException) ושואלים באיזו רמת בדיקה הוא צריך להיתפס לראשונה.
- **מיון פעילויות לשלב בתהליך הבדיקה**: נותנים רשימת פעילויות (ניתוח דרישות, כתיבת מקרי בדיקה, ניתוח פגמים, כתיבת קוד בדיקה וכו') ועל התלמיד למיין אותן בין שלבי STLC (תכנון, עיצוב, מימוש, ביצוע).
- **כתיבת קריטריוני קבלה מסטורי**: נותנים סיפור משתמש נתון, והתלמיד מתבקש לנסח קריטריוני קבלה (לפעמים באמצעות פורמט Given-When-Then).
- **טריאז' ריצת CI אדומה**: מתארים תרחיש של צינור CI שנכשל (למשל בדיקת GUI גויסה, הגדרת נתונים לא תואמת) והמשימה היא לזהות אם זה באג מוצר, באג בבדיקה או בעיית סביבה.
- **סיווג סוגי בדיקות**: מציגים סדרת תיאורי בדיקה (למשל “בדיקת עומסים של שירות API”) והתלמיד מסווג אם היא פונקציונלית/לא-פונקציונלית, Unit/Integration/System וכו'.
- **תרגיל על עקרונות הבדיקה**: מציגים תרחיש כמו “בודק מריץ את אותם המבחנים כל פעם” ושואלים איזה עיקרון נגזר מזה (לדוגמה Pesticide Paradox במקרים שחוזרים רק על אותם מקרים בלי חידוש).

סגנון שאלות פרק 2 (מלכודות מסיחים)

בחינת ISTQB בפרק זה נוטה להשתמש בשאלות רב-ברירה עם תשובות מבלבלות: למשל

- הסחות דעת בין הגדרות קרובות (למשל להציג תשובה על שני מודלים ולהעביר אותה כבוחן, או להשתמש במילים נרדפות של “בדיקת מערכת/בדיקת אינטגרציה”).
- מושגים דומים שעטופים מנוסחים מתחום אחר (לדוגמה “בדיקת smoke” או “ביצוע SRA” כאילו הם סוגי רמות).
- שימוש במשפטי כזב “All of the above” או “None of the above” כדי לבדוק הבנה עמוקה (למשל, “מריצים Unit tests לפני Integration tests – נכון/לא נכון?”).
- הסחות משמעותיות: מידע לא רלוונטי בסיפור התרחיש (כמו פירוט ארוך של תצורה), או מספר אופציות נכונות ואי אפשרות בחירה מרובה (מאתגר הבחנה מדויקת).
- רבדים של שאלה אחת (“איזה מהם אינו תפקיד הבודקת ב-Agile?”) שמערבבים תחומים (למשל תפקיד בודק מול הבדל רמות בדיקה).
- שימוש במונחי עברית/אנגלית אקדמיים (למשל swap functional עם user story, או שימוש ב-“בדיקת הפתיחה Regression”).

הקשר ישראלי

- בראיונות עבודה בישראל נפוצות שאלות כמו "מה ההבדל בין בדיקות פונקציונליות לרגרסיה?" או "מה עושים בתהליך בדיקה בפרויקט Agile?". לעתים מצפים למונחים בשתי השפות: למשל "Functional Testing (בדיקות פונקציונליות)", "User Story (סיפור משתמש)", "Acceptance Criteria (קריטריוני קבלה)".
- מונחים מקובלים בעברית: "בדיקות יחידה" ל-Unit Tests, "בדיקות אינטגרציה" (לעיתים מפורטות ל-Component/ System Integration), "בדיקות מערכת", "בדיקות קבלה" (לפעמים נאמר UAT – קבלת משתמש). "קביעת מוכן/ביצוע", "סיפור משתמש", "הגדרת סיפוק" (פחות בשימוש, לרוב אומרים Definition of Done).
- בדיקות Agile (סקורמים): ראו דגש על "סיפור משתמש", "Sprint", "Retrospective-I Review", וכמובן CI/CD (צנרת פיתוח רציפה). בישראל אוהבים גם להתייחס ל-ATDD/BDD (לדוגמה: "ניסוח Given-When-Then לאותם קריטריונים").
- מקורות ישראליים: לדוגמה באתרי קהילת בדיקות וברשת (מגזין "עולם הבדיקות", בלוג Ein-Des-Ein, שאלות תרגול בעברית) מזכירים לעתים שאלות כמו "תסביר מה זה Shift-Left", "מה זה בדיקת הוכחה לקונפירמציה לעומת רגרסיה". ההיבט הפרקטי הוא שמראייני QA מצפים להכיר מונחי יסוד הן באנגלית והן בעגה המקצועית בעברית, ויודעים לשאול על "קונטקסט Agile" גם בשפה הטכנית וגם בשפה היומיומית.

מקורות

מקור (קישור)	מפרסם, שנה	רישיון
-Four Types of Shift Left Testing [42†L100 L109] [46†L277-L285]	,Carnegie Mellon SEI 2020	All rights) Proprietary (reserved)
-ISTQB CTFL v4.0 Syllabus [45†L198 L207] [54†L222-L228]	ASTQB (ארה"ב), 2018	Proprietary
-The V-Model in Software Testing [49†L47 L54] [49†L74-L81]	Qt Company, 2022	Proprietary
ולוג גרעיניים (ISTQB Level Info) – Test Institute [L39-L43†56]	,ISTQB Foundation Blog 2016	Proprietary
-L386†13] UAT – Deep Dive (עברית) [64†L1400-L1404]	ZAPTEST, 2023	All rights reserved
בדיקת אלפא / בטא – ZapTest (עברית) [18†L239-L247] [L183-L191†18]	ZAPTEST, 2023	All rights reserved
בדיקת בטא – ZapTest (עברית) [L239†18 L247] [111†L1-L4]	ZAPTEST, 2023	All rights reserved
בדיקות אלפא – ZapTest (עברית) [L183†18 L191] [106†L1-L4]	ZAPTEST, 2026	All rights reserved
Black vs White Box Testing – PractiTest [69†L101-L106] [L87-L91†69]	PractiTest, 2024	All rights reserved
What is Confirmation Testing – QA.World [L26-L32†72]	QA.World, 2024	All rights reserved
-What is Regression Testing – IBM [75†L14 L20]	IBM, 2026	Proprietary

רישיון	מפרסם, שנה	מקור (קישור)
All rights reserved	Agile Alliance, 2023	-ATDD – Agile Alliance Glossary 【77†L383-L390】 【77†L391-L395】
Proprietary	Atlassian, 2024	-Definition of Ready (Atlassian) 【103†L1471-L1479】 【78†L31-L35】
Proprietary	Atlassian, 2024	-Definition of Done (Atlassian) 【80†L1471-L1475】
Proprietary	,GeekInterview (kurtz182) 2023	Role of Tester in Agile (GeekInterview) 【38†L104-L107】 【L83-L92†38】
CC BY-SA 3.0	Wikimedia (Hebrew Wikipedia), 2024	סיפור משתמש (ויקיפדיה) 【L138-L142†95】